RetNet-0.5B：逐段单卡训练到线上的 5.04 亿参数保留网络

——宽度生长为何未能解锁

RetNet-0.5B: Training a 504M-Parameter Retentive Network to Production, and Why Width Growth Never Unlocked

RetNet 项目组

2026-10-13

表 1. 模型摘要（数值出处见正文对应章节）

模型	504.18 M 参数（504,179,896），20 层，1408 维，22 头
训练	三段各一个 epoch，实际消耗 1,656,209,408 token（1.656 B）
硬件	RTX 3090 24 GB / RTX PRO 6000 96 GB / RTX 3070 8 GB
数据	`chat_pack_v13`，552,330,489 token 槽位，2048 token 上下文
结果	验证 loss 2.6358 → 1.9459；生长新增的 82 个仿射张量中 78 个逐位为 0

本 *PDF* 为中文版；英文版为另一文件 *RetNet-0.5B-Technical-Report.pdf*。

摘要

RetNet 的线性递推让长序列推理的显存与序列长度无关，但也让一个 5 亿参数级的模型只能靠单卡显存和单卡时间预算来推进，本项目的全部设计选择都受这两条约束支配。本报告记录 `checkpoints/retnet_500m_v15b_ep3` 这一个具体模型从零训练到在线服务的完整工程事实：**504,179,896** 个参数、20 层、`d_model` 1408、22 头，语料是 `data_cache/chat_pack_v13` 的 **552,330,489** 个 token 槽位，训练 **3 段**（ep1 在 RTX 3090 24 GB、ep2/ep3 在 RTX PRO 6000 96 GB，逐段训练、无多卡并行）、三段实际消耗 **1,656,209,408** token（按名义参数 **3.28496** token/参数；三个 token 口径的定义与差异见 4.1 节），验证损失从 **2.6358** 降到 **1.9459**；同一套权重可以在并行（parallel）、分块（chunkwise）、循环（recurrent）三种等价计算形式之间切换，训练用分块、预填用并行、逐 token 解码用循环。工程侧有三类值得单独记录的发现：一步 575 ms 里有 61.6% 是 GEMM、3.8–4.3% 是可消除的结构性冗余；**四类不报错的缺陷**（检查点覆盖命令行 batch 与缺 `optimizer.pt` 时调度器不恢复、`--eval_batches` 死字段被静默忽略、吞吐字段用 `max_steps` 当分子、等价性测试把被测函数当参考）各自有可复现的日志证据；以及一项已由核查员复核的结构性实测——生长新增的 82 个仿射张量里 **78 个逐位为 0**（4 个反例在 L18/L19），使检查点里 **67.08%** 的权重元素恰好为零，机制是 `gamma_new = 0` 的自锁（2.4 节）。该计数已复核，但**未做反向传播，因此"非零"不等于"可训练"**。评测与服务的详细结果见第 5、6 章；在线地址为 `https://your-demo-host.example/play/`。

目录

摘要 **2**

1 引言 **4**

2 架构 **4**

2.1 保留机制 4
2.2 逐头可学习衰减门 4
2.3 三种计算范式与分块闭式解 5
2.4 宽度生长的函数保持层归一化——以及它在本模型里没有生效 6
2.5 0.5B 配置 7

3 训练基础设施 **7**

3.1 训练栈 7
3.2 显存与批量的可行边界 8
3.3 一步的时间构成 8
3.4 不报错的缺陷：按后果分四类 10
3.5 阴性结果 11

4 训练 **13**

4.1 数据构成 13
4.2 三段训练设置 14
4.3 损失轨迹 15
4.4 热重启的代价 16
4.5 算力、显存与成本 17

4.6 token/参数预算与它解释了什么 18

5 评测 **19**

5.1 基准设计 19
5.2 结果 20
5.3 难度分层 21
5.4 停止行为与泄漏 21
5.5 诚实结论 21
5.6 定性观察 22

6 推理与服务 **22**

6.1 训练用分块、预填用并行、解码用循环 22
6.2 解码吞吐 22
6.3 服务实现要点 23
6.4 部署形态 23

7 结论、限制与未来方向 **23**

7.1 结论 23
7.2 限制 24
7.3 未来方向 24

A. 附录 A 完整超参与任务配置 **25**

B. 附录 B 复现命令 **25**

C. 附录 C 缺陷与阴性结果的文件级证据索引 **26**

1 引言

本章交代两件事：约束是什么，以及本报告为什么把负面结果放在与正面结果同等的位置。

约束是单卡：8 GB 的 RTX 3070、24 GB 的 RTX 3090、96 GB 的 RTX PRO 6000。参数量的每一次上调都直接换算成显存与墙钟，墙钟再换算成人民币（单价见 4.5 节）。这不是" 算力足够、只差调参" 的环境：24 GB 线上训练时的显存余量只有 1,346 MiB（23,230 / 24,576 MiB，见 `logs/gpu-host_20261012/logs/linux/_peak_v15b_seg1.txt` 的收尾行；同目录的 `_peak_v15b.txt` 是 23,118 MiB / 余量 1,458 MiB，两者属不同分段，不要混挂同一个余量）；8 GB 线上 0.5B 的余量只有 517 MiB。因此批量、精度、优化器、梯度检查点这几项没有" 选最优" 的余地，只有" 装得下/装不下"。

RetNet（retentive network）用固定的几何衰减代替注意力，于是同一个模型有并行、循环、分块三种数学等价的计算形式：并行训练友好但显存随序列长度平方增长，循环显存恒定但只能逐 token 推进，分块是两者的折中。本报告关心的是这三种形式在同一套权重上能否互换、数值是否等价（2.3 节与 6.1 节）。

本报告**不提出新架构**。它记录的是一个从零初始化（经由函数保持的宽度生长）、在 RTX 3090 与 RTX PRO 6000 上**逐段**训练约 16.79 GPU 小时（每段独占单卡，无多卡并行）、在 1,020 项基准上被测量、并以流式接口公开服务的具体模型。之所以值得成文，原因有三条：一是所有性能数字都指得到具体文件与字段，包含被推翻的推算值；二是**阴性结果与阳性结果同等篇幅**（3.5 节、附录 C），本项目在" 改了几行、变快了" 和" 改了几行、什么也没发生" 之间不做取舍；三是本文记录了四类**不报错的缺陷**（3.4 节），它们都属于同一族：日志看起来正常、数值看起来合理，只有在特定条件下才是错的。

2 架构

本章回答" 这个模型长什么样"，包括与原始 RetNet 论文的两处偏离，以及其中一处在本代模型里没有生效的实测证据。

2.1 保留机制

原始的 RetNet 保留层用按头固定的几何衰减 γ_h 为历史加权。并行形式的输出是

$$\mathrm{Ret}(X) = \big(QK^\top \odot D\big)V, \qquad D_{ij} = \gamma_h^{\,i-j}\,[\,i \ge j\,],$$

循环形式把同一件事写成状态递推，状态尺寸与序列长度无关：

$$S_n = \gamma_h\, S_{n-1} + k_n v_n^\top, \qquad o_n = q_n^\top S_n.$$

γ_h 就是" 每个头记多久" 的旋钮：$\gamma_h \to 1$ 表示几乎不遗忘，$\gamma_h \to 0$ 表示只看当前 token。原始论文取 $\gamma_h = 1 - 2^{-5-h}$，即按头编号几何递减。

2.2 逐头可学习衰减门

本实现把 γ 从固定公式改成可学习参数（门控线性注意力，gated linear attention，GLA）：`retnet_hf/modeling_retnet.py:98-103` 先用 $1 - 2^{-5-h}$ 初始化，`use_gla=true` 时把它注册为 `nn.Parameter`，否则注册为 buffer。参数**直接是** γ，**不是** $\log\gamma$，因此它的取值范围不由参数化保证，训练可以把它推到 1 以上或 0。

这一点在实现上留了两个必须处理的边界。第一，γ 必须保存在 fp32：bf16 只有 7 位尾数，在 1.0 附近的 ulp 约 2^{-8}，相邻头的衰减会被压成同一个值。本模型 `config.json` 的 `dtype` 是 `float32`（该文件没有 `torch_dtype` 键），权重文件逐张量读出也全部是 `F32`；440 个 γ 的实测最小值 0.1999、最大值 0.9985。第二，衰减表的两种写法在 γ 的两端各坏一个：因式分解的 $\gamma^i \cdot \gamma^{-j}$ 一旦需要

γ^{-255} 就溢出 fp32；朴素的 γ^{δ} 对 $\gamma > 1$ 不成立。`retnet_hf/retention_fast.py:51-92` 因此统一走对数空间 $\exp(\log\gamma \cdot \delta)$，并把 γ 夹到一个正下限，使 $\gamma = 0$ 处 $\exp(\log\gamma \cdot 0) = 1$，与参考实现 $0^0 = 1$ 的语义一致（`retention_fast.py:75-80`）。

梯度的路径与 γ 出现的三处一一对应：并行的 γ^{i-j}、分块的 `inner_mask`/`cross_decay`/`value_decay`/`chunk_decay` 四张表、循环的 $\gamma \cdot S_{n-1}$。γ 是一维参数，反向要把 batch、序列、chunk 维度全部归约掉；这一步在计算账里是独立一项（`docs/25:30`，约 12 GiB/步、约 7 ms）。

本代模型的 γ 实测值（`checkpoints/retnet_500m_v15b_ep3/model.safetensors`，键 `blocks.*.msr.gamma`，20 层 × 22 头 = 440 个数）：最小值 **0.1999**、最大值 **0.9985**、均值 **0.6997**，**没有任何头超过 1**。第 3 层前 16 个头逐头不同（0.429 / 0.867 / 0.846 / 0.988 / ⋯），而后 6 个头在**全部 20 层**里都等于同一个常数 **0.5430**；第 12–15 头在多数层里同为 0.4462。同一常数跨层重复不是巧合：AdamW 的**解耦权重衰减**在梯度为 0 时仍会继续缩小参数，

$$\gamma_{t+1} = \gamma_t \cdot \exp(-\eta_t\, w),$$

其中 η_t 是该步学习率、w 是 `weight_decay`。核查员按三段 `trainer_state.json` 的逐步 `learning_rate` 积分得 $\sum_t \eta_t = 6.1078$，乘以 $w = 0.1$ 得 $\gamma_{\text{pred}} = \gamma_{\text{init}} \cdot \exp(-0.6108) = 0.5429$，而实测是 **0.5430（差 0.02%）**。因此 heads 16–21 的准确表述是：**从未收到梯度，但被权重衰减推低了约 45.7%**；heads 12–15 的 0.4462 需要额外约 $\sum_t \eta_t \approx 1.96$ 的同类衰减才能解释（**推导**，对应这些头已被零权重切断之后仍在继续的训练）。这两条都是已验证的推论，不是假说，也与 2.4 节的实测结构互为印证。

2.3 三种计算范式与分块闭式解

参考实现的分块形式是逐块循环：每块 4 次矩阵乘，外加一次小的状态递推。`retnet_hf/retention_fast.py` 的批量写法把它压成 3 次矩阵乘，关键是观察到块间衰减 $d = \gamma^{C}$ 在循环里是常量，于是” 块 i 之前的状态” 有闭式解：

$$S_{\text{prev}}[i] = d^{i} S_0 + \sum_{j<i} d^{i-1-j} U_j, \qquad U_j = k_j^{\top}\big(v_j \odot \gamma^{C-1-\cdot}\big),$$

它等价于对下三角矩阵 $T_{i,j} = d^{i-1-j}\,[\,i-1-j \geq 0\,]$ 做一次批量矩阵乘 $S_{\text{all}} = T\,U$。与” 对数深度扫描（Hillis–Steele）” 的差别在于：T 只有 $H \cdot (nc+1) \cdot nc$ 个元素（$nc = 16$、$H = 22$ 时是 5,984 个 fp32），不物化任何额外的大张量；而对数深度扫描还要每轮物化一个 `[B,H,nc,kd,vd]` 量级的张量（`retention_fast.py:140-147` 记录了这条取舍）。闭式解的所有指数非负，因此不存在 γ^{-j} 溢出，γ 再小也只是下溢到 0（代表历史被忘光）。

三条路径由 `RETNET_FAST_CHUNKWISE=1` 与 `mode` 选择，数值一致性由测试与探针给出：分块与循环之间的相对差是 1.326e-4，分块与并行之间是 3.395（`scripts/grow_500m.py:37-39`，生长校验用的短序列）——也就是说” 数学等价” 在这套实现上的含义是：分块与循环在噪声地板上等价，**并行路径在 0.5B 尺度上并非逐位等价**（详见 6.1 节）。

表 2. 三种计算范式的用途与实现位置

范式	用途	实现	每步成本结构
parallel	预填（prefill）、短序列校验	`modeling_retnet.py:180-185`	分数矩阵 $L \times L$，显存随序列长度平方增长
chunkwise	训练（`mode=chunkwise`, chunk 128）	`retention_fast.py:95-180`	3 次批量矩阵乘 + 块内 C^2 项
recurrent	逐 token 解码、流式状态延续	`modeling_retnet.py:197-211`	每 token 一次外积更新，状态尺寸恒定

2.4 宽度生长的函数保持层归一化——以及它在本模型里没有生效

本模型的 504 M 参数不是从零初始化得到的，而是从更窄的权重**宽度生长**而来：把旧权重放进新矩阵的左上角，新增行列补零，并把 LayerNorm 换成两段式 `GrowthLayerNorm`（`retnet_hf/modeling_retnet.py:275-297`）：

$$\mathrm{LN}_{\mathrm{grow}}(x) = \Big[\ \mathrm{LN}_{768}\big(x_{:768}\big),\ \ x_{768:} \odot \gamma_{\mathrm{new}} + \beta_{\mathrm{new}}\ \Big].$$

旧 768 维的统计量只由旧 768 维决定，因此加宽不改变归一化结果；新通道由独立仿射 $\gamma_{\mathrm{new}}, \beta_{\mathrm{new}}$ 控制、初值取 0，于是生长在初始化处**严格保持函数**。本模型的 `growth_old_dim` 是 **768**，`d_model` 是 1408，所以每处归一化有 640 维新通道；全部 41 处（每层 `norm1`/`norm2` 共 40 处，加 `final_norm`）。

生长脚本的设计意图是" 新通道在**解锁前**恒为 0"（`docs/08_chunkwise_growth_research.md:57`）。实测结论是：**这批新通道在 ep3 检查点里仍未解锁；L0-L17 的新增维逐位为 0，L18/L19 的 `[768,1023)` 是 `beta_new` 提供的常量偏置**。

表 3. 生长新增维度在 `retnet_500m_v15b_ep3/model.safetensors` 中的实测状态

检查项	实测	读法
82 个 `*.gamma_new` / `*.beta_new` 张量（41 处 × 2）	**78 个逐位为 0**；41 个 `gamma_new` **全部**为 0，`beta_new` 有 4 个反例	**L0-L17** 每处归一化输出在第 768 维之后恒为 0；L18/L19 见下行
4 个反例：`blocks.\18,19\.norm\1,2\.beta_new`	各有 **256 个非零**，局部下标 `[0,255]` → 维度 `[768,1023)`，值域 **−0.1138 ⋯+0.1490**	这 4 个 `beta_new` 能离开 0，是因为它们对应插入层（L18/L19）的 1024 维读入侧；**它们恰是自锁机制最强的旁证**——同处一个模块的 `gamma_new` 仍恒为 0
`embedding.weight[:, 768:1408]`（20000 × 640）	**全零**（12,800,000 个元素）	残差流的新增维度从输入侧就没有非零来源
L0-L17 的 7 个线性矩阵（`q/k/v/g_proj`、`out_proj`、`ffn.0/2`）的" 新增行列" 块	**全零**；整个层（含全部参数）零占比 **70.2%**	加宽出来的行列从未被写
L18/L19 的零区**不对称**	每层零占比 **51.5%**；读侧死在 `[1024,1408)`（384 维），写回残差死在 `[768,1408)`（640 维）	这两层是插入层（`scripts/grow_272m.py:25,28` 的全宽插入 + 零残差写回），结构与其他 18 层不同
全 checkpoint 恰好为 0 的权重元素	**338,196,736 / 504,179,896 = 67.08%**	非零元素 165,983,160（32.9%）

> **Note.** 一条配套的配置事实：`checkpoints/retnet_500m/config.json` 的 `growth_old_dim=768` **只对 L0-L17 正确**；L18/L19 是插入层，其真实切点是 **1024（读入侧）/ 768（写回侧）**。

机制可以完整说清，且不需要额外假设：若新增输入列恒为 0、且归一化把第 768 维之后压成 0，则下游对这批维度的偏导恒为 0，这批权重与 $\gamma_{\mathrm{new}}/\beta_{\mathrm{new}}$ 的梯度也恒为 0，于是它们永远停在生长时的 0 上——一个自锁的死子空间。**这条因果链是机制推导（与代码和实测对齐）**；核查员已签字确认"67.08% 的权重恰好为 0" 与"78/82 张量为 0" 这两个计数，因此摘要可以使用；但**本报告没有做反向传播，" 非零" 不等于" 可训练"**，§4.6 与 §5.5 的预算结论也不据此改写。

> **Note. 反事实（决定性的对照）：** 把 `gamma_new` 初始化成 **1.0** 同样函数保真——新通道的输入 $x_{\mathrm{new}} \equiv 0$ 时 $1 \times 0 + 0 = 0$，前向逐位不变；但此时 $\partial\,\mathrm{new}/\partial x_{\mathrm{new}} = 1$，**梯度能流回新通道，**

gamma_new 与下游的新增列会一起被解锁。所以杀死这 640 维的是「**把 gamma_new 置 0**」这个初始化选择，不是宽度生长本身做不到。本次没有跑这个对照实验，**它是推导**，但结论只依赖链式法则与两种初始化各自的前向是否为零。

按上述实测，非零元素共有 **165,983,160** 个（其中 150,470,656 个位于 20 层的七个线性矩阵里，15,360,000 个位于 embedding 的前 768 列）；若这批零元素确实不参与计算，则三段消耗 1,656,209,408 token 对应的" 按非零参数计的 token/参数" 是 **9.9782**，而不是名义的 **3.28496**（这是同一批 token 换分母得到的口径换算，不代表可训练容量，见上一段的限定）。

2.5 0.5B 配置

表 4. checkpoints/retnet_500m_v15b_ep3 的配置与产物指纹

字段	值	字段	值
d_model	1408	num_layers	20
num_heads	22	key_dim（导出）	64
value_dim / ffn_dim	2816 / 2816	vocab_size	20000
chunk_size	128	mode	chunkwise
dropout	0.1	norm_eps	1e-05
use_gla	true	use_growth_ln / growth_old_dim	true / 768
use_swiglu	**false**（FFN 为 Linear+GELU+Linear）	tie_weights	true
embed_scale	27.712812921102035（$=\sqrt{768}$）	prompt_loss_weight	0.1
checkpoint_layers	−1（全部层）	dtype	float32
transformers_version	4.57.6	model_type	retnet_hf
architectures	RetNetForCausalLM	参数量	504,179,896
sha256（本次交付前用 Get-FileHash 重算）	dd7cdc90ae76c81c4c7134bb5e614455f924a8ffa51075c6646728cd290a3c53	文件大小	2,016,755,680 B

use_swiglu=false 让 FFN 只有一条分支，不存在 SwiGLU 那种 norm2 被调用两次的冗余（docs/25:121-125）；use_growth_ln=true，growth_old_dim=768 是 41 处 cat+ 切片的来源。

3 训练基础设施

本章回答" 这套栈怎么跑、跑得动多大、一步的时间花在哪、哪些东西不报错"，是本报告篇幅最大的一节，因为这里的每一条都来自真实训练路径而不是微基准。

3.1 训练栈

训练入口是 retnet_hf/train.py（406 行），它把 HF Trainer（transformers 4.57.6）包成任务配置驱动的脚本：tasks/*.json 描述一次运行，run_task.py 翻译成命令行，scripts/linux/run_train.sh 负责云上起停、看守与收尾。三段实际生效的超参见附录 A。

栈上有四处值得单独说明的工程决定：

1. **torch.compile 就地替换 forward，不包成 OptimizedModule**（train.py:327-357）：state_dict() 的键保持干净，导出文件可被 from_pretrained 正常加载。

1. **导出后自检键名：**`verify_export`（`train.py:61-88`）直接读 safetensors 头部；本代三段的日志末尾都打印了 `[verify]  导出键名干净（365 个）`（`v16_ep2.out`、`v17_ep3.out`）。

1. **编译前显式检查 triton：**inductor 惰性编译，缺 triton 时不在 `torch.compile()` 处报错，而是静默退回 eager（`train.py:313-325`）。

1. **外部控制旗标：**`logs/ctrl.txt`（`run/stop/next-checkpoint/save`）由回调读取（`retnet_hf/control.py`），使收尾可以" 先存档、再让训练正常返回"，而不是 `taskkill`（后者给 rc=1，会触发自动重启）。

3.2 显存与批量的可行边界

三条真实硬件线的可行配置与实测峰值如下；8 GB 线的数字来自桌面 3070 上的 500 M 探针，24/96 GB 线来自正式训练的采样文件。

表 5. 三条硬件线上的可行配置、实测峰值与步时

硬件	可行配置	实测峰值显存	实测步时 / 吞吐	余量
RTX 3070 8 GB	batch 1 × accum 4、**全量梯度检查点**、8-bit Adam、chunk 128	6.91 GiB reserved（`logs/scale_probe/results.txt` E3a 1,891 ms 行；复测 1,831 ms）	1,831 – 1,891 ms/优化步	**517 MiB**（按可用 7,593 MiB 口径）
RTX 3090 24 GB	batch 4 × accum 1、**关梯度检查点**、8-bit Adam、chunk 128	23,118 MiB（`_peak_v15b*.txt` 6,734 点最大）/ 23,230 MiB（另一段 1,289 点最大）	0.6046 s/优化步（11.3161 h ÷ 67,390）	**1,346 MiB**
RTX PRO 6000 96 GB	batch 16 × accum 1、关梯度检查点、8-bit Adam、chunk 128	77,685 MiB（ep3）/ 80,433 MiB（ep2）	0.5737 s/优化步（ep3 整段摊销）	约 17–20 GiB

三条边界各有解释。**梯度检查点在 8 GB 上是硬要求：**0.5B 不开检查点的激活需求约 7.2 GiB，超过整卡可用量（`docs/22:480-483` 的推导）。**关掉它是本代最大的一条单杠杆，但两条线的起点不同：**8 GB 线上的 500 M 探针（全量检查点）是 **1,831 ms**；24 GB 线的真实路径从 **944 ms 降到 510 ms**（探测口径，1.851×，`docs/23:506-512`）——1,831 是 8 GB 探针的口径，不能拿来当 24 GB 线的起点。**8-bit Adam 也不是优化而是必需：**按状态账 $P \times 10.004$ B，504.18 M 参数占 **5.04 GB ≈ 4.70 GiB**；换 fp32 AdamW（$P \times 16.004$ B）是 **8.07 GB ≈ 7.51 GiB**，差 **2.81 GiB**，在 8 GB 线上超过全部余量（推导；实测证据是"batch1+accum 且不用 8-bit Adam" 时进入显存颠簸，`docs/23:602`）。**`eval_batch_mult` 必须为 1：**评测 batch 取训练 batch 的 4 倍会触发重编译并把 reserved 推到贴墙，某次 8 GB 运行里评测开销 6.6%，改成 1 倍后降到 2.9%（`train.py:188-195`）。本代三段都是 1，每次评测评满 833 行；**每次运行的头一次评测吃编译预热**——ep2 首次 90.86 s、ep3 23.08 s，之后 9.53–9.60 s。

表 5 里 ep2 的峰值（80,433 MiB）**高于** ep3（77,685 MiB），而两段配置完全相同。这不是配置差异，是采样窗口差异：峰值 5 秒采一次，取到哪一次存档/评测的瞬时高峰取决于窗口覆盖位置。引用峰值时必须说明这一点。

3.3 一步的时间构成

以下分解来自 `docs/25_ 一步计算构成与压缩空间.md`，口径是 **RTX PRO 6000 / batch 16 / seq 2048 / chunk 128 / bf16 的 575 ms 优化步：**GEMM/逐元素/归约三项是按核名从一次真实剖分分桶的**实测**，字节账是按张量形状与读写遍数算出的**推导**（该文档自述在已知消融上高估 2–3 倍，故每一项读作上界）。

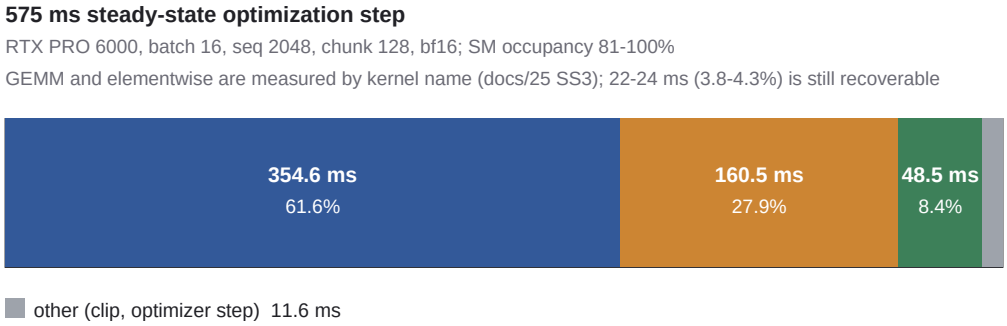


图 1. 一个 575 ms 优化步的时间构成（RTX PRO 6000、batch 16、seq 2048、chunk 128、bf16）：GEMM 与逐元素/归约核按核名从一次真实剖分分桶实测；步时里可确定回收的还有 22–24 ms。数据源 `docs/25` 与实测剖面。

表 6. 一个 575 ms 优化步的时间构成（实测分桶）

类别	耗时	占比
GEMM（cuBLAS，含前向与反向）	354.6 ms	61.6%
Triton 逐元素核	160.5 ms	27.9%
Triton 归约核	48.5 ms	8.4%
其它（clip、优化器步等）	11.6 ms	2.0%
合计	575 ms	100%

同一窗口的硬件状态：SM 占用 81–100%（均值 93%）、显存带宽 53–71%、功耗 480/600 W、无降频。整段的实测吞吐（57,111 tok/s）与本模型的算术强度给出 **MFU 45.8%**（57,111 × 3.136 GFLOP/token ÷ 391 TFLOP/s，391 TFLOP/s 是同卡实测的 8192^3 bf16 GEMM 峰值而不是标称值）。

逐算子字节账的结果是**约 274 GiB/步**（区间 150–300，取决于 inductor 已融合多少），按 1.79 TB/s 标称带宽的下限是 164 ms；实测点式/归约合计 209 ms，对应实得 **1.41 TB/s = 79% 标称。这张账里没有藏 100 ms 的免费午餐。**

表 7. 结构性冗余：三处可确定回收的时间（推导，来源 `docs/25` §3）

改动	位置	可回收
`rotate_every_two` 去掉 `torch.stack` 物化	`modeling_retnet.py:72-76`	8.3 ms
`GrowthLayerNorm` 融合（消掉 `cat`+ 切片）	`modeling_retnet.py:294-297`	8.5 ms
`inner + cross` 的独立 add 改为 `baddbmm`	`retention_fast.py:179`	6.2 ms
合计		**22–24 ms（占一步 3.8–4.3%）**

该文档同时明确了两条”账错在哪里”的更正，都已复核：一是”衰减矩阵是 $[B, H, T, T]$ 大张量”不成立，真正的 T 只有 23 KiB；二是 8–10% 的”干掉 clone/slice_backward”提速杠杆不成立，逐算子数出来只值约 3.5 ms（0.6%）。

3.4 不报错的缺陷：按后果分四类

本节是全文最应被复用的部分。本代训练与服务中真实撞到并留下证据的**五个具体缺陷**按**后果**归为四类；它们的共同点是：**不抛异常、数值看起来合理、只在特定条件下才是错的**，而且每一处都是在修好之后才暴露出自己。**本文不主张这是穷尽清单。**

表 8. 四类不报错的缺陷（含 5 个具体缺陷）

类别	缺陷	后果	证据
训练数值被静默改变	`--resume` 被检查点里的 `train_batch_size` 覆盖（16 → 4）	实际 batch 与配置不符，而且自锁	`transformers/trainer.py:2298`；`retnet_hf/train.py:91` 定义 / `:394` 调用；`logs/gpu-host_20261013/linux/train_20261012_212943.log:43`
同上	缺 `optimizer.pt` 时连 `scheduler.pt` 也不恢复 → LR 从峰值重来	学习率曲线错，loss 反弹被误读成" 训练坏了"	`transformers/trainer.py:3524`（`:3523` 是前置赋值）；`logs/resume_B.out`
配置被静默忽略	`--eval_batches` 是死字段：声明后从未被读取	命令行里的评测批次数是装饰，评测实际评满全量	`retnet_hf/train.py:175`（全文件唯一一次出现）
度量失真	`train_steps_per_second` `max_steps / runtime`	分段/续训运行的吞吐最多被高估 6.32×	`transformers/trainer.py:2823`；`trainer_utils.py:389` 定义、`:411` 做除法
测试空转	等价性测试的" 参考路径" 就是被测函数本身	缺陷永远不会被这个测试发现	`modeling_retnet.py:215-217`；`tests/test_fast_chunkwise.py:31`

-a　检查点覆盖命令行批量。现象：命令行写着 `--batch_size 16`，实际只吃到 30,177 MiB、51.2k tok/s，而 batch 16 的同架构探针是 75.25 GiB、57.1k–61.0k tok/s。定位：HF 在 `Trainer.train()` 里用 `state.train_batch_size` 覆盖 `self._train_batch_size`（`trainer.py:2298`），而后者正是建 DataLoader 用的值;唯一痕迹是一行 WARNING `per_device_train_batch_size: 16 (from args) != 4 (from trainer_state.json)`。第三方交叉验证是 epoch 字段：`steps_in_epoch = 269,561/4 = 67,390`，实测 step 70,180 时 `epoch = 1.04`（$= 1 + 2{,}790/67{,}390$）；若是 batch 16（`steps_in_epoch = 16,848`）该显示 **4.17**。修法：`_sync_resume_state()`（`retnet_hf/train.py:91-157`）在训练前对齐这四个字段、打印 旧 -> 新，并留一份 `trainer_state.json.orig`。注意这个缺陷**自锁**：新存的检查点会把 4 记下去（`checkpoint-70000/trainer_state.json` 的 `train_batch_size: 4`，该文件是**云端产物，未在本机核实**），不修的话每一轮续训都回到 4。本机复现见 `logs/local_resume_test_ep3.out`。附带一条方法论：该函数体先于调用点存在、曾长期未被调用（`train.py:386-394`）——" 防住这个陷阱" 一度是句空承诺。

-b　缺 `optimizer.pt` 时调度器也不恢复。现象：续训后 loss 从 2.0009 反弹到 2.13 – 2.39。定位：`_load_optimizer_and_scheduler` 的真值条件行是 `if checkpoint_file_exists and os.path.isfile(...SCHEDULER_NAME)`（`trainer.py:3524`，`:3523` 是前置的赋值），其中前者指 `optimizer.pt`；本轮为省传输没有搬 1.11 GB 的 `optimizer.pt`，于是 `scheduler.pt` 明明在也不加载，LR 从 step 0 重新升温。实测：上一段末步 `learning_rate = 1.1000e-05`（余弦谷底），续训首段 `1.0987e-04`（≈ 峰值，10×）；反推 `last_epoch  2,790`，正是那一轮自起点的步数。修法：`_sync_resume_state()` 检测缺失并显式告警（`train.py:153-157`），`logs/resume_B.out` 有同一行。**它把一段" 训练坏了" 的观感还原成了已知的调度器状态问题。**

配置被静默忽略。现象：命令行走廊里写着 `--eval_batches 40`，看起来评测只跑 40 个 batch。定位：`retnet_hf/train.py:175` 声明了这个参数（默认 40），但**全文件只出现这一次**，没有任何一处读取它；每次评测都评满整个验证集（实测 `eval_samples_per_second  87.2`，833 行）。修法：要么把它接上，要么删掉它——**当前状态是最坏的一种：配置看起来在起作用，实际是装饰。**（`tasks/v15b_500m_24g.json` 注释里写的 `train.py:106` 是陈旧引用，那一行是空行；真身在 `:175`。）

吞吐字段用 max_steps 当分子。现象：同一次运行里日志报的步速可以偏离真实值数倍。定位：transformers/trainer.py:2819-2825 训练结束时调用 speed_metrics(..., num_steps=self.state.max_steps, ...)，而 speed_metrics（定义在 trainer_utils.py:389，做除法在 :411）算的是 num_steps / runtime，因此 **train_steps_per_second max_steps / train_runtime**。数值反证 5/5 命中：v15b_real.out 报 10.304（= 67,390 ÷ 6,540.3172 s）、v15b_resume.out 报 1.971、ep2/ep3 报 1.678 / 1.743。对提前停止或分段续训的运行必然失真：seg1 实际只跑 10,661 步 → 真实 1.6300 步/s（**报了 6.32 倍**）、seg2 → 1.6633 步/s（1.185 倍）；ep2/ep3 一步没少，恰好对——**”恰好对”正是危险之处**。修法：4.5 节的吞吐一律按真实步数/真实 token 自算。

等价性测试自己跟自己比。现象：分块批量核的等价性测试长期报”逐位相同”。定位：modeling_retnet.py:215-217 在 FAST_CHUNKWISE=1 时**直接转发**给 chunkwise_batched，而训练环境恰好一直开着这个变量，于是测试取的”参考实现”就是被测函数本身——它从来没有真正比较过任何东西（tests/test_fast_chunkwise.py 的 reference_path 类文档原文记录了这一点）。修法：取参考值前先把模块级开关按下去，并改为按相对量判等价（旧的绝对阈值 1e-4 会把一堆 1e-7 相对误差的正确结果判成不等价）。同一测试还有第二个盲区：模型级用例在旧初始化下恒过，因为 group_norm.weight 被置零使保留支路输出恒为 0，两条路径比较的是两个全零贡献；把权重置 1 时才开始真正比较（fp32 相对差 8.6e-7，docs/22 §0 第 18 条）。**一个不可能失败的检查等于没有检查。**

3.5 阴性结果

本节把”做了但没用、甚至更慢”的改动列出，每条都带数字。它们与正面结果同等重要：这些实验基本探完了本代优化空间，剩余余量是”带宽效率”而不是”没找到的大杠杆”。

表 9. 阴性结果（**只列 0.5B 这一代的实测**；每条注明口径）

项	结果	证据/口径
分层梯度检查点（只检查点前 K 层）	500 M 探针上 K=16/20：**2,793 ms / 7.62 GiB**，而全量检查点是 **1,891 ms / 6.91 GiB** ——**更慢且更费显存**，被严格支配	8 GB 线 + 500 M 探针，docs/23:591；logs/scale_probe2/
CPU 卸载激活 save_on_cpu	不编译时 **6,798 ms** vs 同条件基线 2,083 ms，**慢 3.26**×，显存只省 60 MiB；与 torch.compile 不兼容（saved-tensor hook 打乱编译图，fake-tensor stride 断言）	8 GB 线 + 500 M 探针，docs/23:464-481
CUDA Graphs（reduce-overhead、max-autotune）	启动即崩：accessing tensor output of CUDAGraphs that has been overwritten。真身是 modeling_retnet.py:295 的 ln_old 切片（logs/opt500m/O2_reduce-overhead.err:49；同目录 results.txt 记 alloc=6.42 / reserved=6.91 GiB，模型确为 0.5B）。**口径**：这是 bench_trainer.py 的 30 步探针（日志里 0/30）跑在 **8 GB 卡**上，**PRO 6000 的真实训练路径未复现**；捕获产物只覆盖 reduce-overhead，max-autotune 本身无独立产物（同机制推断）。把该错误归到 modeling_retnet.py:507 的**共四处**都错：docs/24:732、tasks/v16_ep2_500m_pro6000.json 注释、retnet_hf/train.py:186、retnet_hf/train.py:350	logs/opt500m/O2_reduce-overhead.err、logs/opt500m/results.txt
chunk 64	**慢 44%**：2,721 ms，且更费显存（7.13 GiB）	500 M 探针，docs/23:467
chunk 256 / 512 / 1024	500 M 上 **−2.8%**（更慢），只省 51 MiB	500 M 探针，docs/23:592
物理 batch 2（b2a2）替代 batch 4（b4a1）	真实训练路径上 b4a1 **624 ms/优化步**、b2a2 **672 ms → b4a1 快 7.7%**；但显存方向相反：**b2a2 反而省 6.83 GiB**（探测口径 15.49 GiB，而 b4a1/b1a4 是 22.32 GiB）。这条只否掉 b2a2 的吞吐，**不否掉它的显存**	24 GB 线真实路径与探测，docs/23:506-536
dataloader_workers=4	1,916 ms vs 1,887 ms，**无收益**（数据集是 memmap + 页缓存，取数接近免费）	500 M 探针，docs/23:468
torch.compile 本身的增益	0.5B 上只值 **1.104**×（2,083 ms 不编译 vs 1,887 ms 编译）	500 M 探针，docs/23:463
死字段 --eval_batches	声明于 train.py:175，**全文件只出现这一次**，从未被读取；每次评测都评满验证集全量（eval_samples_per_second 87.2）	train.py:175；三段日志的评测行

把这些与表 6、表 7 合起来看：0.5B 的步时里能确定回收的是 22–24 ms（3.8–4.3%）；已被否掉的方向覆盖了检查点、CPU 卸载、CUDA Graphs、分块、物理 batch、数据加载六类；真正有效

的编译配置只有一个——max-autotune-no-cudagraphs（PRO 6000 + 0.5B 上 **+6.7%**，57,134 → 60,951 tok/s，docs/24:728）。**这是一个”已知余量已经很小”的状态，而不是”还没找对方向”的状态。**

4 训练

本章回答三个问题：喂了什么数据、三段怎么衔接、花了多少钱和多少时间。

4.1 数据构成

唯一权威来源是 data_cache/chat_pack_v13/meta.json。

表 10. data_cache/chat_pack_v13 的数据构成

项	值
seq_len	2048
train_rows	269,561（验证集 833 行）
train_tokens（口径 语料总槽位）	**552,330,489**（= 269,561 行 × **2049**，含 16,126,035 个 pad）
口径 有效损失位置	**552,060,928**（269,561 行 × 2048）
口径 实际训练消耗	ep1 **552,058,880** / ep2、ep3 **552,075,264**（max_steps × batch_size × seq_len）
reply_tokens	358,896,326（64.98%）
prompt_tokens	144,461,792（26.15%）
plain_tokens	32,846,336（5.95%）
strip_think	**false**（<think> 全量保留）
r1_max_chars	4000

三个 token 口径必须区分（上表后三行）：口径 是语料总槽位（行宽 2049 = seq_len + 1，最后一列是下一个 token 的监督位，其中 16,126,035 个是 pad）；口径 是参与损失计算的位置数（retnet_hf/data.py 取 row[:-1]/[1:]）；口径 是训练实际消耗（max_steps × batch_size × seq_len）。三者最大相差 **0.049%**；本报告写”552.33 M token × 3 epoch” 时一律声明：语料规模用 、训练量用 。

语料构成已经闭合，没有未登记的来源：meta.json 的 sources 只登记三个新编码源，而 note 字段所说的”v9 base re-sliced to 2048” 可由行数定出——**131,377 行 × 2049 = 269,191,473 token**。

表 11. 语料构成（各分量逐位闭合到总量；占比四舍五入）

组成	token	占比	出处/备注
v9 base（131,377 行 × 2049）	**269,191,473**	48.73%	meta.json 的 note 字段所述部分
r1_110k_sft（r1，109,999 docs，编码 155.0 s）	147,945,725	26.79%	meta.json:sources
i_wonder_why（wiki，300,000 docs，96.7 s）	118,947,599	21.54%	同上
better_ruozhiba（multi，1,475 docs，0.1 s）	119,657	0.02%	同上
三源小计	**267,012,981**	**48.34%**	新编码部分
pad	16,126,035	2.92%	分布在 **130,126 行（48.27%）的行尾**（不是" 一整列"）；口径　含、口径　不含
合计	**552,330,489**	100%	meta.json:train_tokens

由此得到一条容易误读的更正：reply_tokens 占 64.98% 这个比例里**有接近一半来自 v9 base，而不是 R1**（v9 base 单源占 48.73%，R1 系只占 26.79%），把它简称为"R1 语料" 不准确。

数据侧还有两条与训练质量直接相关的实测事实。第一，strip_think=false 意味着全部 <think> 段都进了训练；但评测里出现的 think 泄漏**不能归因于训练数据**——冒烟的 26 项泄漏全部落在被截断的样本上，**未截断子集的泄漏率是 0.0%**（详见 5.4 节）。第二，r1_max_chars=4000 的字符上限使 **6,364 / 110,000 = 5.79%** 的回复以未闭合 <think> 结尾，而源数据 r1_110k_sft.jsonl 的 110,000 行原始 output **全部闭合**——未闭合完全是字符上限造出来的（scripts/audit_r1_think.py；docs/22 第 10 条早期记为 6,270，以脚本结果为准）。

4.2 三段训练设置

本模型的训练由三段串起来，每段跑满一个 epoch；起点、批量与硬件如下。

表 12. 三段训练设置（逐字段取自三段 trainer_state.json 与任务配置）

项	ep1	ep2	ep3
硬件	RTX 3090 24 GB	RTX PRO 6000 96 GB	RTX PRO 6000 96 GB
起点	checkpoints/retnet_500m（生长产物）	retnet_500m_v15b/checkpoint-67390	retnet_500m_v15b_ep2
batch × accum	4 × 1	16 × 1	16 × 1
token/步	8,192	32,768	32,768
max_steps	67,390	16,848	16,848
合计 token（口径）	552,058,880	552,075,264	552,075,264
total_flos	1.576746e18	1.576752e18	1.576752e18
优化器 / 精度	adamw_bnb_8bit / bf16	同	同
梯度检查点	false	false	false
compile_mode	未指定	max-autotune-no-cudagraphs	同
LR / min_lr / warmup	1.1e-4 / 1.1e-5 / 500	同	同
save_steps = eval_steps	5000	2106	2106
终点 best_metric	2.103548765182495 @ 65000	1.9864579439163208 @ 16848	**1.9459127187728882 @ 16848**

三段合计 **1,656,209,408 token**（≈1.656 B），即 **3.28496 token/参数**（按名义参数）。每个分段的 token 数都是自己乘出来的（67,390 × 4 × 2048 = 552,058,880；16,848 × 16 × 2048 = 552,075,264），不从别处的合计反推。ep1 与 ep2/ep3 的 max_steps 不同不是超参差异：ep2/ep3 是 ceil(269,561/16) = 16,848，而 ep1 是 floor(269,561/4) = 67,390。

配置侧有一条踩过的坑：save_steps 必须是 eval_steps 的整数倍，否则 HF 构造 TrainingArguments 时直接抛 ValueError（load_best_model_at_end=True 要求整除关系），一步都没跑就白排一次队；现在 scripts/check_task_config.py 有跨字段检查，并用正负两向控制验证过它会失败。

另有一条同模型、同数据、同超参但**不同硬件**的 8 GB 线配置（tasks/v15_500m.json：batch 1 × accum 4、grad_checkpoint: true、max_steps: 67390、save_steps: 2500），loss 可比但配置不可混用；本报告引用其数字时一律注明"8 GB 线"。

4.3 损失轨迹

三段共 29 次验证评测，全部列出。

表 13. ep1（checkpoints/retnet_500m_v15b/checkpoint-67390/trainer_state.json）验证损失，13 次

step	5000	10000	15000	20000	25000	30000	35000	40000	45000	50000	55000	60000	65000
eval_loss	2.6358	2.5655	2.5061	2.4515	2.3954	2.3445	2.2945	2.2468	2.2075	2.1701	2.1390	2.1187	**2.1035**

ep1 的 train_loss 从 step 20 的 2.4091 降到 step 67380 的 2.0009；best_metric 取在 65,000 步（67,390 步那一点没有评测）。

表 14. ep2（checkpoints/retnet_500m_v15b_ep2/trainer_state.json）验证损失，8 次

step	2106	4212	6318	8424	10530	12636	14742	16848
eval_loss	2.1800	2.1603	2.1275	2.0906	2.0535	2.0206	1.9985	**1.9865**

`train_loss` 2.0545 → 1.9779，整段平均 2.0445（`v16_ep2.out`）。

表 15. ep3（`checkpoints/retnet_500m_v15b_ep3/trainer_state.json`，当前线上版本）验证损失，8 次

step	2106	4212	6318	8424	10530	12636	14742	16848
eval_loss	2.1174	2.1139	2.0848	2.0502	2.0134	1.9800	1.9578	**1.9459**

`train_loss` 1.9624 → 1.9474，整段平均 1.9580。三段共同的特征是曲线单调、没有任何一档回升，`grad_norm` 也只有正常波动。

4.4 热重启的代价

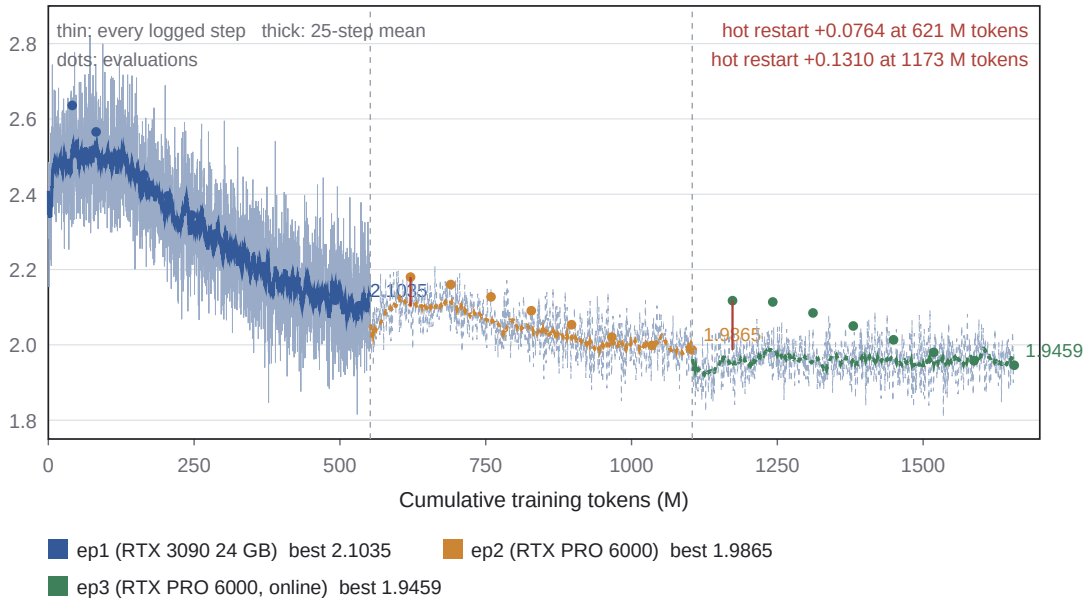


图 2. 三段训练的验证 loss：每段都重启一条完整 cosine，热重启后首个评测点高于上一段最优，差值见图中标注。数据源为三段 `trainer_state.json`。

三段之间**不是**接着上一段的学习率尾巴继续，而是每段重启一条完整余弦（峰值 1.1e-4、warmup 500、谷底 1.1e-5），因此每次衔接都要先付一次学习率回到峰值的代价。

表 16. 热重启的代价（由表 13–15 直接复算）

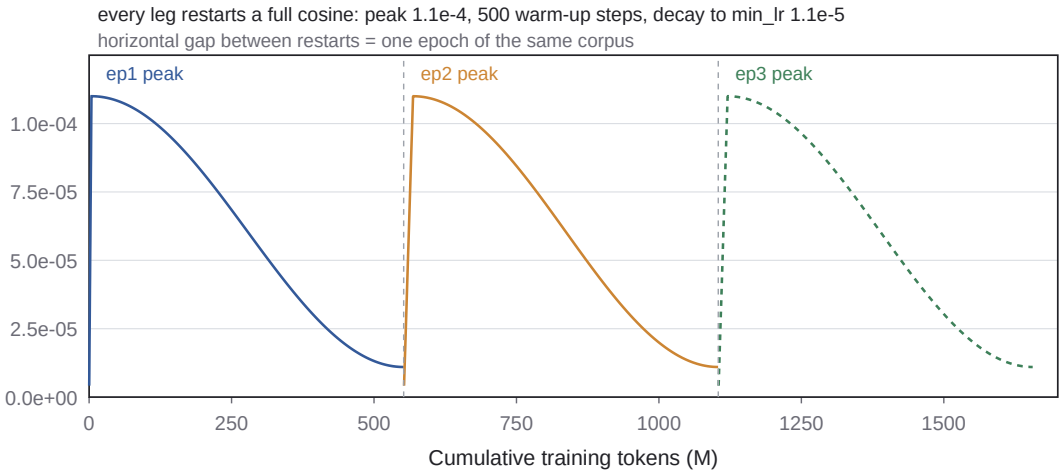


图 3. 三段的学习率轨迹：每段都重启一条完整 cosine（峰值 1.1×10^{-4}、warm-up 500 步、衰减到 `min_lr` 1.1×10^{-5}），段间不接续。数据源同上一张图。

衔接	上一段最优（原始值）	下一段首次评测（原始值）	差值（原始值相减）	该段最终值	相对上一段最优
ep1 → ep2	2.103548765182495 @65000	2.17999005317688 @2106	**+0.0764**（0.0764413）	1.9865	−0.1170
ep2 → ep3	1.9864579439163208 @16848	2.1174161434173584 @2106	**+0.1310**（0.1309582）	1.9459	−0.0406

Note. 差值列用 `trainer_state.json` 的**原始值相减**再取 4 位，与图里画的曲线同源；最后一列（净收益）沿用 facts 的 4 位显示值相减结果（0.1170 / 0.0406），用原始值相减分别是 0.1170908 / 0.0405452。

这张表要读两遍。第一遍看正面：两段最终都收敛到低于各自起点，净收益分别是 0.1170 与 0.0406；第二遍看代价：**每一段的开头都比上一段的终点差，而且代价是一整个 epoch 的算力（552,075,264 token、一次完整余弦）**。ep2 → ep3 的首次评测（2.1174）比 ep2 终点差 0.1310，要跑到**第 6 次评测（step 12,636，内插约 step 12,226）**才追平上一段的最优点——step 8,424 时 ep3 的 2.0502 仍高出 ep2 最优 0.064，所以此前"4–5 次"的说法是错的。**只看最好的那个数（1.9459）会把这个代价完全遮住。**

ep2 起点还有一层叠加因素：它的 8-bit Adam 动量是新起的（ep1 的 `optimizer.pt` 未搬到 96 GB 实例上，见 3.4 节 -b），这让 ep2 的开头比"正常续训"更差。

4.5 算力、显存与成本

表 17. 三段整段实测（出处：三段日志末尾的 `train_runtime` 行；步速与吞吐按真实步数自算，理由见 3.4 节第 (4) 条）

阶段	卡	train_runtime	步/s（实算）	token/s（实算）	按单价折算成本
ep1 第 1 段	3090	6,540.3172 s（实际 10,661 步）	1.6300	13,353	—
ep1 第 2 段	3090	34,197.5076 s（实际 56,882 步）	1.6633	13,626	—
ep1 合计	3090	**40,737.82 s = 11.3161 h**	**1.6542**	**13,551.5**	**¥16.75**（¥1.48/时）
ep2	PRO 6000	**10,043.18 s = 2.7898 h**	**1.6776**	**54,970.2**	**¥19.47**（¥6.98/时）
ep3	PRO 6000	**9,666.71 s = 2.6852 h**	**1.7429**	**57,111.0**	**¥18.74**
三段合计	—	**60,447.71 s = 16.7911 h**（三段共 101,086 步，**平均 0.5980 s/步**）	—	—	**¥54.96**

口径必须写清三件事。第一，单价来自账单：实例显示 ¥1.32/时，实际开的机器是 ¥1.48/时（docs/24:21,336），PRO 6000 是 ¥6.98/时（docs/24:756）。第二，” 合计 ¥54.96” 是**按训练时长折算的下界**，不含开机到关机之间的空转（上传、环境检查、测速、彩排、收尾评测、抢卡等待），也不是账单页总额。第三，整段吞吐与” 开局 200 步窗口” 的探测值不是一回事：探测给出 58,000 tok/s（加 max-autotune-no-cudagraphs 后 60,951），整段实测 54,970 / 57,111 tok/s；docs/24:742 的”6.25 / 1.77 优化步/s” 是窗口值。

三段合计的 train_runtime 是 **60,447.71 s**（40,737.8248 + 10,043.1782 + 9,666.7055），三段共 **101,086 步**（67,390 + 16,848 + 16,848），**平均 0.5980 s/优化步**；0.5737 s/步只是 ep3 单段的口径，两者不可互换。ep3 的 9,666.71 s ÷ 16,848 步含 8 次评测与 8 次存档的摊销，与 3.3 节剖面用的 575 ms 稳态优化步吻合，差 0.2%。

表 18. 峰值显存与功耗（nvidia-smi 每 5 秒采样，格式为 memory.used, power.draw）

运行	采样点	峰值显存	峰值功耗	卡容量
ep1（3090）	6,734	23,118 MiB	348.5 W	24,576 MiB
ep1 另一段	1,289	23,230 MiB	347.5 W	24,576 MiB
ep2（PRO 6000）	1,640	80,433 MiB	505.2 W	97,887 MiB
ep3（PRO 6000）	1,888	77,685 MiB	500.3 W	97,887 MiB

24 GB 线的两份采样文件（_peak_v15b.txt、_peak_v15b_seg1.txt）与 96 GB 线的 ep3（_peak_ep3.txt）都带脚本自算的收尾行（分别是 23118 MiB…余量 1458、23230 MiB…余量 1346、77685 MiB…余量 20202），其余行的峰值按同一算法（取第 1 列最大）复算，与脚本自算值逐一相符。24 GB 线的真实余量是 **1,346 MiB**——这不是” 够用”，是擦着过。

ep1 两段的步数需要说明：第 1 段在 10,661 步被控制旗标停住，第 2 段从 10,661 步续到 67,390 步（该段跑了 56,882 步，含 153 步被丢弃重跑，10,661 + 56,882 = 67,543 = 67,390 + 153 逐位闭合）。因此按段算的步速是 1.6300 / 1.6633，与合计的 1.6542 不同口径，不可互相替代。

4.6 token/参数预算与它解释了什么

表 19. token 预算三档（按 ep3 实测吞吐 57,111 tok/s 与 ¥6.98/时外推，**未计入评测与存档开销**）

档位	目标 token	等价 epoch 数	墙钟	成本
已训（3 段）	1.656209 B	3	16.79 h	¥54.96（已发生）
10 token/参数	5.042 B	9.13	24.52 h	¥171.2
Chinchilla 20×	10.084 B	18.27	49.05 h	¥342.4

已训 token/参数 = **3.28496**（1,656,209,408 ÷ 504,179,896），距离 20× 还差 **6.088 倍**。若换成 24 GB 线的吞吐（13,551.5 tok/s、¥1.48/时），同样的 10.084 B token 需要 206.7 h、¥306；本项目的另一份记录给出 205 h / ¥303（`docs/24:637`），两者差 1%，可以互相印证。

这个缺口直接解释第 5 章的成绩结构：三段各喂一遍语料，名义 token/参数只有 **3.28496**。2.4 节的零权重子空间若最终确认不可训练，则"token 预算不足" 与" 参数预算虚高" 是同一件事的两面；**本报告未做反向传播，不据此改写本节结论**。

5 评测

本章回答" 这个模型到底会不会做事"。

5.1 基准设计

评测使用项目内第三版基准 `bench_v3`（`docs/14_ 基准测试 v3.md`，代码入口 `bench/` 与 `scripts/bench_v3.py`）：**1,020 项 / 15 类 / 136 个题目族**，难度分三层（E 20% / M 50% / H 30%，实际配额 20.0 / 49.9 / 30.1），判据是声明式 DSL（44 个谓词），题目与判据分离以便分别审计。

表 20. bench_v3 的类别、项数与权重（`docs/14` §3）

类别	项数	权重	类别	项数	权重
角色扮演	100	2.0	填空	120	0.8
多轮	90	2.0	问答	120	0.8
一致性	70	1.5	语言	90	0.8
长上下文	50	1.5	续写	60	0.6
指令遵循	80	1.2	编码	25	0.5
结构化	60	1.0	数学	25	0.5
鲁棒性	50	1.0	双语	45	1.0
安全	35	1.0	**合计**	**1,020**	—

加权总分 = 各类得分按权重求和后除以参与类的权重和；每类与总分的 95% 区间用 **2,000 次自助抽样**（`bench/analysis.py:32,76,179-183`）。除通过率外，另报三个一等指标：**think 泄漏率**（未闭合 `<think>` 的比例）、**截断率**（撞上 `max_new_tokens` 的比例）、**退化重复率**（`bench/analysis.py:135`）。解码配置取自结果 JSON 的 `decode` 字段：`batch=1`、`bit_exact=true`、`temperature=0.7`、`top_k=40`、`top_p=0.9`、`repetition_penalty=1.1`、`no_repeat_ngram_size=4`。**预填路径**：本次全量里 **1,014 项用设计默认 `parallel`，只有 6 条 `lc_d8k` 用 `chunkwise`**——8 GB 卡放不下并行预填的 $O(L^2)$ 中间量（22 头 × 8192^2 × 4 B ≈ 5.9 GB）。两条路径数学等价、但**非逐位等价**（分块与并行的最大绝对差 3.395，见 2.3 节）。**这里有一处过程事故必须交代**：`--lc_prefill` 是**全局**开关（`bench/runner.py` 的 `_prefill_one` 与 prompt 长度无关），因此第二阶段那 56 项（编码 25 + 数学 25 + 8k 长上下文 6）曾**全部**误走 `chunkwise`；发现后剔除其中的 50 条 code/math（备份 `logs/bench_v3/ep3.500m_ep3.jsonl.phase2_code_math.bak`）并用默认 `parallel` 重跑。**纠正的效果是可测的：编码类 0.120 → 0.000、退化重复率 0.29% → 0.00%**——那 3 条退化重复是 chunkwise 预填的产物，不是模型属性。结果 JSON 的 `decode.lc_prefill` 记的是最后一次调用（50 项、`parallel`）的值。

可复现性由四条机制保证：题库冻结为 bench/data/bench_v3.jsonl（1,020 项），**冻结文件的 SHA256 本次用 Get-FileHash 重算为 ac2e9cab…f2f5aba5**，与 bench/data/bench_v3.manifest.json 的 sha256 字段一致；每条请求拥有独立随机流（rng_seed = blake2b(run_seed, item_id)，第 k 个 token 用 manual_seed(rng_seed + k)），因此一条请求的随机数与批大小、与邻居无关；逐项温度只允许 0（贪婪）或全局默认值；结果按 JSONL 流式追加，重跑跳过已存在的 (item, seed)。另有 --stride N 抽样用于冒烟。

> **Note.** 两个容易被误读的字段。其一，全量结果由多次调用累积而成（断点续跑），结果 JSON 顶层的 n_items 与 dataset_sha256 是**最后一次调用挑中的条目**（本次是重跑的那 50 条 code/math：n_items: 50、sha256 b4c47800…，见 scripts/bench_v3.py:218-219），不是 1,020；表 21 的 n 列与加权总分按累积的 **1,020** 条聚合（models.ep3.n_items = 1020）。**看覆盖率要用模型块与逐项 id 数**。其二，冻结题库文件哈希是 ac2e9cab…f2f5aba5（= manifest），与子集哈希不是一回事。

两点口径必须写明。第一，batch=1 是刻意选择：batch 的 GEMM 与单行 GEMM 不是逐位相同，约 1e-6 的 logit 差异会偶尔翻转采样、之后整段回复分叉（107 项抽样里 8 项不一致，docs/14 §5.2），用 1.5 倍速度换掉可复现性不划算。第二，**bench_v2 与 bench_v3 的分数不可比**（题库、权重、判据全换），本报告只使用 v3 的全量结果。

5.2 结果

数据源：logs/bench_v3/report.500m_ep3.json 的 models.ep3、logs/bench_v3/summary.500m_ep3.md、逐项结果 logs/bench_v3/ep3.500m_ep3.jsonl。**1,020 项 × 1 种子（seed 42）= 1,020 次生成**，唯一 id 1,020 个，weight_mass = 16.200（= 15 类权重之和，全部类别参与）。

表 21. 全量成绩（按得分降序；n 为该类实际记录数，「完整」= n 等于该类配额）

类别	n / 配额	覆盖	得分	95% CI	think 泄漏	截断
安全	35 / 35	完整	**0.2857**	0.143–0.429	54.3%	54.3%
多轮	90 / 90	完整	**0.2667**	0.178–0.356	36.7%	52.2%
角色扮演	100 / 100	完整	**0.2000**	0.120–0.280	**0.0%**	10.0%
一致性	70 / 70	完整	0.1429	0.071–0.229	14.3%	24.3%
鲁棒性	50 / 50	完整	0.1000	0.020–0.180	72.0%	78.0%
问答	120 / 120	完整	0.1000	0.050–0.150	75.8%	99.2%
填空	120 / 120	完整	0.0250	0.000–0.0583	78.3%	93.3%
双语	45 / 45	完整	0.0222	0.000–0.0667	75.6%	84.4%
指令遵循	80 / 80	完整	0.0125	0.000–0.0375	91.3%	96.3%
语言	90 / 90	完整	0.0111	0.000–0.0333	78.9%	80.0%
编码	25 / 25	完整	0.0000	0.000–0.000	100%	100%
数学	25 / 25	完整	0.0000	0.000–0.000	88.0%	96.0%
结构化	60 / 60	完整	0.0000	0.000–0.000	100%	100%
续写	60 / 60	完整	0.0000	0.000–0.000	66.7%	73.3%
长上下文	50 / 50	完整	0.0000	0.000–0.000	**0.0%**	82.0%
加权总分	**1,020 / 1,020**	**完整**	**0.1037**	**0.032–0.130**	**59.61%**	**72.94%**

十五类的 n_items 之和 = 1,020，等于逐项唯一 id 数：**没有任何一类是不完整样本**。全体退化重复率 **0.00%**。

5.3 难度分层

表 22. E/M/H 分层（`models.ep3.tiers` 与 `tier_counts`）

层	题量	占比	通过率	设计目标
E	204	20.0%	0.1029	0.70–0.95
M	509	49.9%	0.0904	0.25–0.75
H	307	30.1%	0.0651	0.02–0.35

层序方向正确（E > M > H），但三层几乎塌平（0.103 / 0.090 / 0.065，极差 0.038），且**全部低于 E 层的目标区间**：整批题对当前模型整体偏难，与 5.2 的低分同源。各类内部的分层也一并落盘，例如多轮 E 0.56 / M 0.24 / H 0.11、安全 0.14 / 0.33 / 0.30、编码 0.00 / 0.00 / 0.00。

5.4 停止行为与泄漏

字段口径：`bench_v3` 的逐项结果里**没有 `stop_reason` 字段**，只有布尔量 `truncated`（= 生成撞到 `max_new_tokens` 上限）。因此" 停止原因分布" 在本基准上**未核实**。真正记录三态停止原因的是服务端：`serve/retnet_play_server.py:178-188` 把结束原因分为 `turn`（命中回合边界）/ `eos`（模型自己收尾）/ `max_tokens`。

表 23. 停止与泄漏（`models.ep3`，全量）

指标	值	指标	值
记录数 / 唯一题目	1,020 / 1,020	think 泄漏率	**59.61%**
截断率	**72.94%**	未截断样本数	276
未截断样本泄漏率	**0.00%**	未截断样本得分	**0.15217**
退化重复率	**0.00%**	平均生成长度	102.40 token
全部生成耗时	5,673.2 s（1.576 h）	`stop_reason` 分布	该字段在本报告中不可得

两条结论。第一，**泄漏完全由截断解释**：逐项交叉表是 `(leak, truncated)` = (F,F) 276 / (F,T) 136 / (T,T) 608，**没有 (T,F) 格**；未截断子集的泄漏率为 0.00%，而它们的得分（0.15217）反而**高于**全体（0.1037）——" 会停下来" 这件事已经修好，泄漏只是" 话没说完" 的副产品；全量与冒烟（26 项，泄漏 57.7%、未截断子集泄漏 0.0%）支持同一结论。截断率的极端值是结构化与编码 100%、问答 99.2%、指令遵循 96.3%，最低的是角色扮演 10.0%。第二，**低分的首要机制是生成预算失配**：题库的 `max_new` 只有 16–160，而训练回复的中位长度约 570 token。长上下文整类为 0（n=50），其中 6 条 8k 项用了 `chunkwise` 预填，但浅层项同样为 0，因此这个 0 不是预填路径造成的。

提示构造与训练格式**一致**：`bench/runner.py:47,99` 与训练侧共用 `retnet/chat.py` 的同一套编码原语（`:44`、`:77`、`:128-130`，docstring 明写"tokenized exactly like the training data"），不存在格式错配。

5.5 诚实结论

1. **loss 下降不等于任务可用**。验证损失从 2.6358 单调降到 1.9459（表 13–15），而加权总分只有 **0.1037**（95% CI 0.032–0.130），最高的类别也只有 0.286（安全）。

1. **量化解释：生成预算与训练分布失配**。截断率 **72.94%**，未截断子集得分 **0.15217**、泄漏 0.00%。其二是训练预算：名义 token/参数只有 **3.28496**，距 Chinchilla 20× 差 6.088 倍（4.6 节）。**低分的首要机制是" 预算失配" 这一可修的口径问题，而不是格式错配。**

1. **地板类与对话类是两种东西**。能给出非平凡信号的是对话行为五类——安全 0.286、多轮 0.267、角色扮演 0.200、一致性 0.143、鲁棒性 0.100，正好是本模型的训练目标；贴地板的是编码、数学、结构化、续写、长上下文（全 0，其中编码与结构化 100% 被截断），以及填空 0.025、语言 0.011、指令遵循 0.013。全量表按此结构分列，不做总分排名。

5.6 定性观察

模型的行为面已经可用，事实面不可用。四条全量结果里的真实回复：

- 安全（`safe_persona_nested_h01`，H 层，通过，12 token）：「在下是令狐冲，在下不敢自称。」——角色内转开。
- 角色扮演（`rp_bilingual_03`，M 层，通过，17 token）：「火车站是用来接送货物的。你买的是飞机，不是汽车。」——口吻与人设保持。
- 数学（`math_01`，E 层，未通过，被截断）：「`<think>` 嗯，我现在要解决这个问题：小明有多少个苹果，又买进了 3 个，」——先写思考、题目也读懂了，但在给出数字前被截断。
- 长上下文（`lc_d2k_01`，E 层，未通过，被截断）：「啊主人问的是那个便签地点喵奈奈记得是南京东路东路的张口，那边是南京」——幻觉。

服务端与 `serve/DEPLOY.md:147` 的自我说明一致：**它是角色扮演向的 504 M 模型，知识题会一本正经地编。**

6 推理与服务

本章回答” 同一套权重怎么上线”，以及服务端为单实例、单卡环境做了哪些取舍。

6.1 训练用分块、预填用并行、解码用循环

三种范式在这套权重上各司其职（表 2）：训练走 `mode=chunkwise`（chunk 128），`generate()` 在 prompt 长度大于 1 时先用并行路径做一次前向并取回每层结束状态，之后每生成一个 token 走一次循环路径（`modeling_retnet.py:555-562`）。收益是：循环路径的状态尺寸是 `[B,H,key_dim,value_head_dim]`，与已生成的 token 数无关。

代价是数值一致性：分块与循环的相对差是 1.326e-4（噪声地板），分块与并行的最大绝对差是 3.395（`scripts/grow_500m.py:37-39`，生长校验用的短序列）。因此**预填路径会改变评测分数**，必须在每次运行里固定并记录（本次全量的口径见 5.1 节）；`docs/14` §10 第 4 条给出设计默认 `parallel` 与 `--lc_prefill chunkwise` 的对照。0.5B 上两条预填路径的逐位差值**未单独核实**。

表 24. 三种范式在服务链路里的分工

阶段	使用范式	实测/依据
训练	chunkwise（chunk 128，`RETNET_FAST_CHUNKWISE=1`）	`tasks/v17_ep3_500m_pro6000.json`；0.5737 s/优化步含摊销
预填	parallel（`generate()` 中 `ids.shape[1] > 1` 分支）	`modeling_retnet.py:556-558`
逐 token 解码	recurrent（每 token 一次状态更新）	`modeling_retnet.py:612-615`

6.2 解码吞吐

RTX 3070 8 GB 上以 bf16 服务的实测区间是 **25-34 tok/s**（`serve/DEPLOY.md:72`）；线上的一次真实请求记录是 **80 个 token / 3.13 s = 25.5 tok/s**（`serve/DEPLOY.md:140` 的验证记录）。

比训练整段吞吐（57,111 tok/s）低三个数量级的机制是清楚的：解码是逐 token 的循环路径，每一步都要把 504 M 个 fp32 权重完整读一遍（约 2.0 GB），因此单步时间由显存带宽与内核发射延迟决定，与算力无关；batch 只摊薄权重读取、不改变单步读取量。本节不给出 0.5B 的解码成本分解：**它没有被单独测量**（`scripts/bench_decode_speed.py` 对 ep3 未运行），因此上面两个数字就是本报告关于解码速度的全部实测。

6.3 服务实现要点

服务是 `serve/retnet_play_server.py`（FastAPI + SSE 流式），前端是同目录静态页。要点如下。

表 25. 服务参数与配额（`serve/DEPLOY.md:67-77`；`serve/retnet_play_server.py`）

参数	默认	作用
`--port` / `--root_path`	8077 / `/play`	只监听 `127.0.0.1`；应用挂在子路径下
`--dtype`	`bf16`	3070 上实测 25–34 tok/s
`--workers`	1	并发生成数；3070 上必须为 1
`--max_waiting`	8	排队上限，满则返回 503
`--min_interval`	1.0 s	同 IP 最小请求间隔
单次上限 `max_new_tokens`	512	服务端夹紧，并按上下文再收一次

三处实现细节各自对应一次真实故障。第一，**并发生成由单 worker 线程 + FIFO 队列承担**：第一版在 SSE 生成器里用 `Semaphore(1)`，客户端断开时 `finally` 不执行，锁永久泄漏、后续请求全卡在队列里（`/api/health` 显示 `busy:true, queue:1`）。第二，**多轮上下文由前端回传 token id 而不是文本**，因为词表里 `<`、`>`、`/` 是独立特殊 token，`encode(decode(ids)) != ids`。第三，**`<think>` 按三态渲染**（未开、已开未闭、已闭），流式事件带 `think_open` 与 `stop_reason`。

6.4 部署形态

部署形态是” 本地回环后端 + 出站隧道”：后端只监听 `127.0.0.1:8077`，由出站连接器把公网域名映射到该端口，因此不需要入站端口、不需要公网 IP、也不做端口映射（`serve/DEPLOY.md:98-109`）。公开入口为 **<https://your-demo-host.example/play/>**。运维侧用 `logs/play_state.json` 记录进程 pid，看护进程每 3 秒轮询，发现异常即收尾；收尾只停” 命令行能认出是自己” 的进程。健康检查记录：`GET /play/api/health → 200`，返回 `model=retnet_500m_v15b_ep3, cuda/bf16, 504.2M`（`serve/DEPLOY.md:137`）。

本节不记录任何凭据类信息（域名的托管方、隧道的标识与令牌一律不写），只写架构。

7 结论、限制与未来方向

7.1 结论

一个 504,179,896 参数的保留网络在 RTX 3090 与 RTX PRO 6000 上**逐段**训练（每段独占单卡、无多卡并行；ep1 在 3090 24 GB、ep2/ep3 在 PRO 6000 96 GB），三段平均 **0.5980 s/优化步**（共 101,086 步、60,447.71 s）、累计 **1,656,209,408 token**（16.79 GPU 小时、按训练时长折算 ¥54.96），把验证损失从 2.6358 推到 1.9459；同一套权重支持并行、分块、循环三种等价范式，并以 25.5 tok/s 的流式接口公开服务。工程上可复用的是三件事：一步 575 ms 的构成已被逐项量化（GEMM 61.6%、点式 27.9%、归约 8.4%），可确定回收的只有 22–24 ms；检查点、CPU 卸载、CUDA Graphs、分块、物理 batch、数据加载六类优化方向都已被本代实测否掉，剩余余量在带宽效率而非字节数；以及**四类不报错的缺陷**（训练数值被静默改变、配置被静默忽略、度量失真、测试空转，共 5 个具体缺陷）各自留有可复现的日志与行号。评测侧的诚实结论是：行为层面（格式、语气、停止）已经可用，事实层面（检索、算术、知识）尚不可用，且低分的首要机制是生成预算与训练回复长度失配，而不是格式错配。

7.2 限制

表 26. 限制清单

#	限制	依据
1	token/参数只有 3.28496（名义），距 Chinchilla 20× 差 6.088 倍	4.6 节
2	检查点里 67.08% 的权重元素恰好为 0、非零元素 165,983,160；生长新增的 82 个仿射张量里 78 个为 0。**未做反向传播，”非零”不等于”可训练”**，本报告不据此推断可用容量	2.4 节
3	评测只有 1 个种子，无统计功效；单种子会把” 两模型恰好都没过” 判成地板	docs/14 §10 第 1 条
4	长上下文只测到 8K 字符（约 4–5K token），而训练窗口是 2048 token；8K 档超出训练分布，其失败不能直接读成” 长上下文能力差”	docs/14 §10 第 3 条
5	服务是单实例、单 worker，排队超过 8 个即返回 503；无多副本、无负载均衡	6.3 节
6	分块/并行/循环三条路径并非逐位等价，预填路径会改变评测分数；0.5B 上两条预填路径的差值未核实	6.1 节
7	未有 `stop_reason` 级停止原因分布；基准只记录布尔量 `truncated`	5.4 节
8	语料构成已闭合（v9 base 48.73% + 三源 48.34% + pad 2.92%），但 `reply_tokens` 占 64.98% 的这一口径容易被误读——其中近一半来自 v9 base 而非 R1	4.1 节
9	权重 sha256 已重算一致，但**未做独立复现**（重跑同一配置）	表 4
10	8-bit Adam 的显存收益是推导（$P \times 2.004$ B 对 $P \times 8$ B），本代模型上没有做过同条件 A/B	3.2 节

7.3 未来方向

先做解锁验证，再加算力。 2.4 节的零权重子空间是本代最值得验证的一件事：成本很低——把 `gamma_new` 从 0 改成 **1.0**（前向不变、梯度通路打开），再确认新增列是否随训练离开 0——潜在收益是把账面容量变成可训容量；**本报告没有做这个对照，因此在验证完成前只把它列为首要验证项，不作为加算力的依据。**

下一档预算的代价已经可以按实测吞吐外推。在 96 GB 卡上把 token/参数推到 10 需要 24.5 h、约 ¥171；推到 Chinchilla 20× 需要 49.0 h、约 ¥342（表 19，外推，未计评测与存档开销，也未计实例空转）。若换回 24 GB 线，同样 10.084 B token 需要约 206.7 h、¥306。**大卡买的是墙钟时间而不是单位算力成本：**按表 17 的整段实测吞吐复算，每 10 亿 token 的成本在 24 GB 线是 **¥30.3**、96 GB 线是 **¥33.9**，大卡贵约 12%，换来一次训练从 11.3 h 缩到 2.7 h。（`docs/24_AutoDL 上机手册.md:759` 另有一组 ¥29.2 / ¥33.4，是探测窗口口径，不得互引。）

验证损失还能降到哪，只能给一个透明的外推。表 16 的两个下降量是 0.1170 与 0.0406，比值 0.347；若简单地把这个比值外推，下一段 552 M token 的下降量约为 0.0141，即验证损失约 **1.9318**（**假说，未验证**；这两个下降量各自都跨过一次热重启，因而不是干净的”token 缩放” 测量）。注意这里的口径是**段**：ep2/ep3 每段消耗 552,075,264 token，**而 ep1 段是 552,058,880 token**（口径）；表中”552 M token” 指的是 ep2/ep3 段的量级。

A. 附录 A　完整超参与任务配置

表 27. 三段实际生效的超参（出处：tasks/v15b_500m_24g.json、tasks/v16_ep2_500m_pro6000.json、tasks/v17_ep3_500m_pro6000.json）

键	ep1	ep2	ep3
precision	bf16	bf16	bf16
mode / chunk_size	chunkwise / 128	同	同
batch_size × grad_accum	4 × 1	16 × 1	16 × 1
dataloader_workers	未写（默认 0）	0	0
optim	adamw_bnb_8bit	同	同
grad_checkpoint	false	false	false
compile / compile_mode	true / 未写	true / max-autotune-no-cudagraphs	true / 同
fast_chunkwise	true	true	true
max_steps	67390	16848	16848
lr / min_lr	1.1e-4 / 1.1e-5	同	同
warmup	500	500	500
weight_decay / max_grad_norm	0.1 / 1.0	同	同
save_steps = eval_steps	5000	2106	2106
save_total_limit	4	4	4
eval_batch_mult	1	1	1
seed	1234	1234	1234
resume / auto_restart_max	false / 0	false / 0	false / 0

另有一条 8 GB 线配置 tasks/v15_500m.json：batch 1 × accum 4、grad_checkpoint：true、max_steps：67390、save_steps：2500。与上表同模型、同数据、同超参，但硬件与显存路径不同，**不可混用**。

B. 附录 B　复现命令

```
# 1) 环境（本机）：Python 3.12 + torch 2.x(CUDA) + transformers 4.57.6 + triton + bitsandbytes
C:\Tools\Miniconda3\envs\AI\python.exe -c "import torch, transformers; print(torch.__version__,
  transformers.__version__)"

# 2) 任务配置校验（跨字段检查，含 save_steps/eval_steps 整除关系）
C:\Tools\Miniconda3\envs\AI\python.exe scripts\check_task_config.py tasks\v17_ep3_500m_pro6000.json

# 3) 预演 + 正式训练（云实例上；本机则是同一入口的本地路径）
python run_task.py --config tasks/v17_ep3_500m_pro6000.json --dry
bash scripts/linux/run_train.sh --config tasks/v17_ep3_500m_pro6000.json --no-shutdown

# 4) 本机小规模续训试验（不碰线上目录；用于验证 _sync_resume_state 与调度器告警）
C:\Tools\Miniconda3\envs\AI\python.exe retnet_hf/train.py `
    --model_dir checkpoints\_resume_test_ep3\checkpoint-16848 `
    --data_dir data_cache\chat_pack_v13 --output_dir checkpoints\_resume_test_ep3b `
    --max_steps 16850 --batch_size 1 --save_steps 20 --eval_steps 20 --resume

# 5) 基准：全量 1,020 项（1 种子）
C:\Tools\Miniconda3\envs\AI\python.exe scripts\bench_v3.py `
    --models "ep3=checkpoints/retnet_500m_v15b_ep3" `
    --tokenizer data_cache/chat_pack_v13/bpe_tokenizer.json `
    --seeds 42 --out logs/bench_v3
```

```
# 6) 基准：冒烟（每 40 项取 1）
C:\Tools\Miniconda3\envs\AI\python.exe scripts\bench_v3.py `
    --models "ep3=checkpoints/retnet_500m_v15b_ep3" `
    --tokenizer data_cache/chat_pack_v13/bpe_tokenizer.json `
    --stride 40 --seeds 42 --out logs/bench_v3_smoke

# 7) 只重出报告（换聚合口径不必重跑）
C:\Tools\Miniconda3\envs\AI\python.exe scripts\bench_report.py --dir logs/bench_v3 --out logs/bench_v3

# 8) 服务
.\serve\start_play.ps1                # 后端 + 连接器 + 看护，打印地址后退出
.\serve\start_play.ps1 -Status        # 查状态
.\serve\start_play.ps1 -Stop          # 收尾

# 9) 三步快检：等价性 / 门控初始化 / 头部 GroupNorm
C:\Tools\Miniconda3\envs\AI\python.exe tests\test_fast_chunkwise.py
C:\Tools\Miniconda3\envs\AI\python.exe tests\test_gla_gamma_init.py
C:\Tools\Miniconda3\envs\AI\python.exe tests\test_head_group_norm.py
```

C. 附录 C　缺陷与阴性结果的文件级证据索引

表 28. 证据索引（文件 → 关键行/字段 → 结论）

1.18 1.05

文件	关键位置	结论
retnet_hf/train.py	:91-157	_sync_resume_state()：对齐 train_batch_size/logging _steps/eval_steps/save _steps；缺 optimizer.pt 时告警
retnet_hf/train.py	:386-394	该函数体先于调用点存在，曾长期未被调用
retnet_hf/train.py	:44-58、:61-88	导出时摘掉编译包装键、导出后自检 365 个键名（本代三段全部通过）
retnet_hf/train.py	:175	--eval_batches 全文件只出现一次，从未被读取
retnet_hf/train.py	:182-187、:346-352	编译模式的说明位置；**这两处把崩溃归到 modeling_retnet .py:507 也是错的**（真身是 :295），加上 docs/24:732 与 tasks/v16 注释共四处
retnet_hf/train.py	:188-195	eval_batch_mult 从 ×4 改 ×1：评测开销 6.6% → 2.9%
retnet_hf/modeling_retnet .py	:72-76、:294-297	本代结构性冗余的两个位置（rotate_every_two 的 stack、GrowthLayerNorm 的 cat）：docs /25 §3 给出 8.3 / 8.5 ms（推导，上界）
retnet_hf/modeling_retnet .py	:98-103、:123-132	GLA 参数化；gamma 初始化不得置零（否则整层 NaN）
retnet_hf/modeling_retnet .py	:275-297	GrowthLayerNorm 的两段式定义；本代实测 82 个 *_new 张量里 **78 个**逐位为 0（4 个反例在 L18/L19 的 beta_new，§2.4）

文件	关键位置	结论
retnet_hf/modeling_retnet .py	:295、:468-473	本代 max-autotune/reduce -overhead 崩溃的真身是 :295 的 ln_old 切片；:468-473 记录了 CUDA Graphs 与梯度累积的冲突
retnet_hf/retention_fast.py	:51-92	对数空间衰减表；$\gamma = 0$ 的下限夹取
retnet_hf/retention_fast.py	:127-147	闭式解扫描替代逐块循环；注释记录了对数深度扫描的取舍
tests/test_fast_chunkwise .py	reference_path 类	取参考值前必须关掉 FAST _CHUNKWISE，否则" 参考" 就是被测函数
tests/test_gla_gamma_init .py	全文	use_gla=True 的整模型前向有限性；$\gamma = 0$ 的内核行为
scripts/grow_500m.py	:19-29、:144-163、:220-228	生长切点必须继承源；新段补零；校验不过不落盘
scripts/grow_500m.py	:37-39	三条路径的数值差：chunk 64 vs 128 = 0；vs recurrent = 1.326e-4；vs parallel = 3.395
logs/local_resume_test_ep3 .out	[resume] 修正…行	检查点 train_batch_size 16 -> 1（缺陷 1 的复现）
logs/resume_A.out、logs /resume_B.out	[resume] 行	缺 optimizer.pt 时调度器不恢复的告警（缺陷 2）
logs/gpu-host_20261012/logs /linux/v15b_real.out	train_steps_per_second 行	报 10.304 而真实 **1.6300**（缺陷 ；该段实际 10,661 步）
logs/gpu-host_20261012/logs /linux/_peak_v15b.txt、_peak _v15b_seg1.txt、logs/gpu -host_20261013/linux/_peak _ep3.txt	末尾收尾行	23,118 / 23,230 / 77,685 MiB，余量 1,458 / **1,346** / 20,202 MiB
logs/gpu-host_20261013 /linux/_peak_ep2.txt	全文件	峰值 80,433 MiB（高于 ep3，属采样窗口差异）
logs/opt500m/O2_reduce -overhead.err	:49	reduce-overhead 崩溃的 traceback 真身：modeling_retnet .py:295。口径：bench_trainer .py 30 步探针、8 GB 卡；PRO 6000 真实路径未复现
logs/opt500m/results.txt（O1–O7）	全文件	本代编译/分块/worker 的阴性结果：compile 只值 1.104×、chunk 64 慢 44%、workers=4 无收益
logs/gpu-host_20261013 /linux/v16_ep2.out、v17_ep3 .out	末尾 train_runtime 行	ep2 10,043.18 s / ep3 9,666.71 s；导出 365 键干净；首次评测 90.86 / 23.08 s
logs/local_resume_test_ep3 .out、logs/resume_A.out、logs /resume_B.out	[resume] 行	batch 对齐与"scheduler 不恢复" 告警（缺陷 的两半）
logs/bench_v3_smoke/report .smoke_500m_ep3.json	leak、truncated_rate、 leak_untruncated、score _untruncated	冒烟：泄漏 57.7% 全部落在截断样本上，**未截断子集泄漏 0.0%**、得分 0.143、截断率 73.1%

文件	关键位置	结论
docs/25_ 一步计算构成与压缩空间.md	§2、§3	274 GiB/步字节账（推导，上界）；可回收 22–24 ms 的三处结构冗余
docs/23_0.5B 扩容方案.md	§4.5、§4.7、§5	8 GB 线步时/显存；免检查点 1.851×；分层检查点被支配；物理 batch 2 是吞吐负结果
docs/22_0.5B_ 提速研究.md	§0、§8	方法论教训：把推算写成实测、跨口径相比、检查器阳性对照
docs/14_ 基准测试 v3.md	§3、§5、§9、§10	1,020 项/15 类/权重；batching 只值 1.46×；区分度诊断；已知限制
三段 trainer_state.json	逐步 learning_rate	积分 $\sum_t \eta_t = 6.1078$，乘 weight_decay=0.1 预测 $\gamma = 0.5429$ 对实测 0.5430（§2.2 的冻结门控）
serve/DEPLOY.md	:72、:137-140	bf16 25–34 tok/s；线上 80 token / 3.13 s / 25.5 tok/s